in the sub-pattern. Thus, we have matches of 'hello hello' and 'world world', in the first sub array element of $matches, then each sub-pattern captured in the second ('hello', then 'world'). Note that the contents of the back-reference are overwritten on each match: \1 contains 'hello' until the PCRE engine reaches and matches the byte sequence of 'world', whereupon the former is overwritten with the latter.

There are alternative syntaxes for back-references, and it is also possible to use symbolic names (in conjunction with named sub-patterns). The escape sequence \g can be combined with an integer to create absolute and relative back-references:

```
preg_match_all('/(\w+) \g1/', $text, $matches);
preg_match_all('/(\w+) \g{1}/', $text, $matches);
preg_match_all('/(\w+) \g{-1}/', $text, $matches);
```

All of the patterns above will produce the same output (in $matches) as the first back-reference example. \g1, \g{1} both match the absolute ordinal value of 1 (the first sub-pattern), \g{-1} is a relative back-reference to the nearest sub-pattern to the left. (\g{-2} would match the second nearest to the left, and so on.) To reference a named sub-pattern, the following sequences all achieve the same (?P=name), \g<name>, \g{name}, \g'name', and the \g forms also accept \k as an alternative (\k<name> and so on).

```
preg_match_all('/(?<thing>\w+) \g<thing>/', $text, $matches);
print_r($matches);
```

```
Output:
Array
(
    [0] => Array ...
    [thing] => Array
        (
            [0] => hello
            [1] => world
        )
    ...
```